

# Camouflage-Aware Policy Network (CAPN) for Graph-Based Fraud Detection

Master's Thesis Presentation

**Fady Hassanein**

2026

# Agenda

- 1. The Problem: Fraud Detection in Networks
- 2. CARE-GNN: The Baseline Approach
- 3. Our Contribution: CAPN
- 4. CAPN Module Roles: GNN, RL, and LLM
- 5. The RL Module: How CAPN Learns Thresholds
- 6. LLM Integration
- 7. Experimental Setup
- 8. Results and Analysis
- 9. Contributions and Future Work

# The Problem

Why fraud detection in graphs is challenging

## Online platforms face coordinated fraud:

- Fake reviews manipulate product ratings
- Fraudsters form networks of coordinated behavior
- They share products, star ratings, and timing patterns
- Traditional methods miss these network patterns

## Graph-based approach:

- Model users/reviews as nodes in a graph
- Connect nodes by shared behaviors (relations)
- Use Graph Neural Networks to classify nodes
- Goal: identify fraud vs legitimate nodes

# The Camouflage Problem

Fraudsters actively hide their identity

## Feature Camouflage

Fraudsters mimic legitimate behavior patterns. They write some genuine reviews to appear normal, making their features look similar to real users.

## Relation Camouflage

Fraudsters connect with legitimate users to blend in. They review popular products alongside their targets, creating edges to many real users.

## Label Camouflage

Training data contains mislabeled nodes. Some fraudsters are marked legitimate (or vice versa), teaching the model wrong patterns.

**Result: Standard GNNs aggregate all neighbor information equally. If a fraudster connects to many legitimate users, the fraud signal gets "averaged out" and the fraudster appears legitimate.**

# Multi-Relation Graphs

Multiple views of the same network

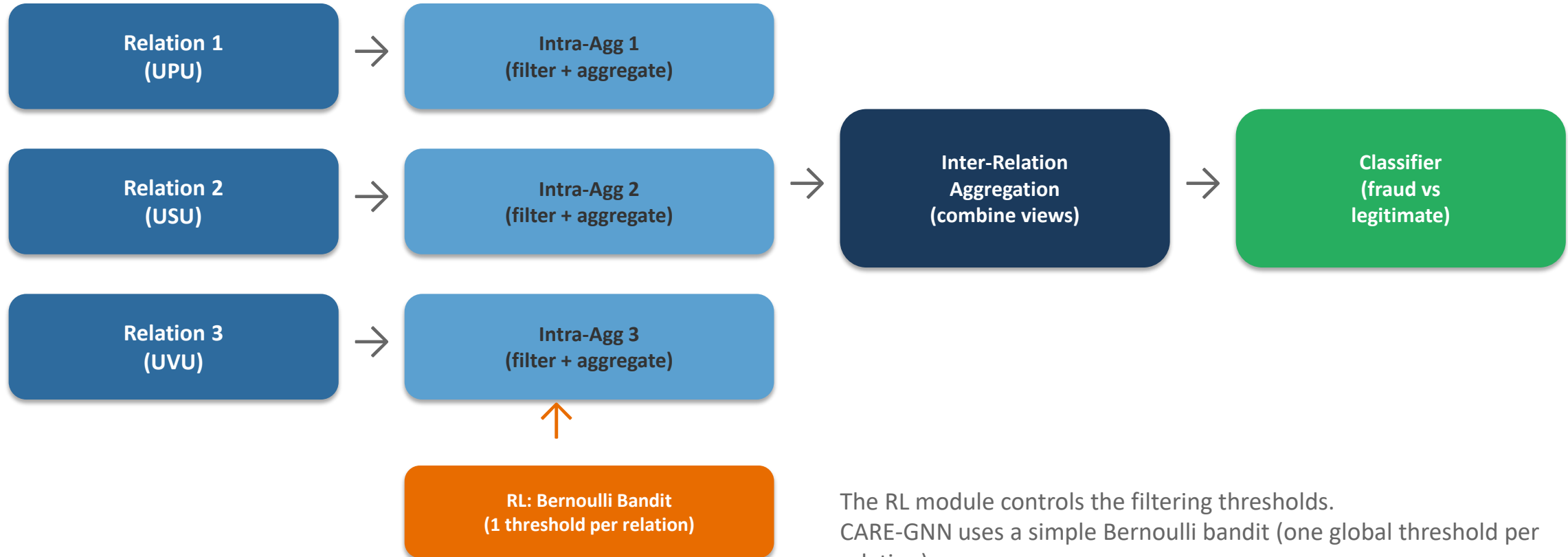
Key insight: Use multiple relation types simultaneously. A fraudster may hide in one relation but expose themselves in another.

| Dataset | Relation 1                  | Relation 2                   | Relation 3                   |
|---------|-----------------------------|------------------------------|------------------------------|
| Amazon  | UPU - Same product reviewed | USU - Same star rating given | UVU - Same reviewer category |
| Yelp    | RUR - Same user wrote both  | RTR - Same time period       | RSR - Same star rating       |

Each relation forms a separate graph over the same nodes. The GNN processes each relation independently, then combines them.

# CARE-GNN Architecture

The baseline: Camouflage-Resistant GNN (Dou et al., 2020)



# Limitations of CARE-GNN

What motivated our work

## Global thresholds

Same threshold for ALL nodes in a relation. A hub node (500 neighbors) uses the same filter as a peripheral node (3 neighbors).

## Stateless RL

The Bernoulli bandit does not look at individual nodes. It cannot adapt to local graph structure.

## Binary reward

Only knows "accuracy up" or "accuracy down". No signal about which nodes or relations improved.

## Linear classifier

The auxiliary label predictor is a single linear layer with limited capacity to learn complex fraud patterns.

# Our Solution: CAPN

Camouflage-Aware Policy Network

**Core idea: Replace the heuristic RL with a learned neural policy that makes per-node, per-relation threshold decisions based on local context.**

| Component       | CARE-GNN (Original)             | CAPN (Ours)                         |
|-----------------|---------------------------------|-------------------------------------|
| RL Algorithm    | Bernoulli bandit (heuristic)    | REINFORCE (policy gradient)         |
| Thresholds      | 1 global per relation (3 total) | Per-node, per-relation (unique)     |
| Action Space    | Up/down by fixed step           | Sample from Beta(alpha, beta)       |
| State           | None (stateless)                | Features + confidence + structure   |
| Reward          | Binary (accuracy up/down)       | Shaped (distance + accuracy + reg.) |
| Label Predictor | Linear layer                    | MLP (hidden layer + ReLU)           |

*Analogy: CARE-GNN is a thermostat (one temperature for the building). CAPN is smart room sensors (personalized temperature per room).*



# CAPN: Three Integrated Modules

Each module has a distinct role

## GNN — The Learner

Role: Classify each node as fraud or legitimate

- Aggregates filtered neighbor features <sup>thresholds</sup> per relation (intra-aggregation)
- Combines 3 relations into one embedding (inter-aggregation)
- Passes through MLP label predictor to get fraud probability <sup>reward</sup>

Output: Fraud probability + confidence score fed back into RL state

## RL — The Controller

Role: Decide how many neighbors GNN sees

- Builds state vector per node: features, <sup>scores</sup> confidence, degree, distances, LLM scores
- Neural network outputs Beta( $\alpha$ ,  $\beta$ ) params
- Samples threshold  $t \in [0, 1]$  per node
- Receives shaped reward from GNN output
- Updates via REINFORCE policy gradient <sup>priors</sup>

Output: Filtering threshold per node per relation (~35,000 on Amazon)

## LLM — The Advisor

Role: Inject fraud domain knowledge (one-time preprocessing)

- Priors: Claude reads dataset structure, sets initial alpha/beta biases for RL policy — warm start before training
- Reasoning Scores: per-node structural profile → 6 risk assessments (JSON) stored as  $[N \times 6]$  tensor, fed into RL state every batch

Output: Priors +  $[N \times 6]$  risk scores tensor

Flow: LLM priors warm-start the RL policy before training. Each batch: RL reads LLM scores in the state vector → outputs thresholds → GNN filters neighbors using thresholds → classifies nodes → accuracy & distance feed back to RL as reward → RL improves.

*Analogy: LLM is the tutor (advice before training). RL is the study strategy planner. GNN is the student.*

# The RL Loop in CAPN

How the policy learns to set thresholds



↻ Update policy weights using  
REINFORCE:  $\text{loss} = -\log\_prob(\text{threshold}) \times \text{reward}$

This loop runs for every batch in every epoch. Over time, the policy learns:

- High-degree nodes with low confidence need stricter filtering
- Nodes with mixed neighborhoods benefit from tighter thresholds
- Peripheral nodes can afford looser thresholds

# Beta Distribution for Thresholds

Natural exploration-exploitation trade-off

The policy outputs alpha and beta parameters for a Beta distribution over  $[0, 1]$ . The threshold is sampled from this distribution.

## Early Training

$\alpha = 1, \beta = 1$

Uniform distribution  
Explores all thresholds equally  
The policy is "trying everything"

## Mid Training

$\alpha = 3, \beta = 2$

Slightly skewed toward higher values  
Starting to favor what works  
Still exploring alternatives

## Late Training

$\alpha = 10, \beta = 3$

Concentrated near optimal threshold  
Exploiting the best value found  
High confidence in decision

**Why Beta? It naturally lives on  $[0,1]$  (perfect for thresholds) and its shape evolves from exploration to exploitation as the policy learns - no manual schedule needed.**

# Shaped Reward Function

Rich learning signal for the policy

$$\text{Reward} = 0.5 \times \text{Distance} + 0.3 \times \text{Accuracy} + 0.2 \times \text{Regularization}$$

## Distance (50%)

Did the threshold make fraud nodes more separable from legitimate nodes in the embedding space?

Uses Multi-View Distance metric across all 3 relations.

## Accuracy (30%)

Did the batch classification accuracy improve?

Same signal as CARE-GNN but computed per-batch for finer granularity.

## Regularization (20%)

Penalty for extreme thresholds (near 0 or near 1).

Prevents collapse to trivial solutions: "filter everything" or "keep everything".

vs CARE-GNN: Binary reward only knows "accuracy up/down" for the entire epoch. Shaped reward tells the policy WHY a threshold was good or bad.

# Supporting Components

MLP Label Predictor & Multi-View Distance

## MLP Label Predictor

- Replaces CARE-GNN's single linear layer
- Architecture: input -> hidden -> ReLU -> output
- More capacity to learn complex fraud patterns
- Provides confidence scores for the state vector
- Result: +5.20pp Label AUC improvement on Amazon

## Multi-View Distance

- Measures fraud/legitimate separability across relations
- Each relation has a learnable weight (gamma)
- Higher gamma = relation is more important for fraud
- Provides the distance signal for shaped reward
- Adapts over training as the GNN improves

# LLM Integration

Using Large Language Models for fraud detection

## v1: Sentence Transformer (Failed)

Approach: Convert node statistics to text, encode with sentence-transformer (384d), project to 16d, concat to state.

Result: No improvement (-0.07pp Amazon)

Why it failed: Lossy round-trip.  
The text encoder treats "degree 23" and "degree 150" as similar strings, destroying the precise numerical relationships.

## v2: Claude Reasoning Scores (Works)

Approach: Send node stats to Claude API, receive 6 structured risk scores (JSON), directly concat to state vector.

Result: +0.06pp AUC, +0.27pp AP

Why it works: LLM reasons about pattern interactions, not just embedding.  
"High degree + high disagreement + low overlap = coordinated fraud"

**Key insight: The value of an LLM is in reasoning about patterns, not in embedding text.**  
**Direct numerical reasoning scores preserve semantic understanding without information loss.**

# Claude Reasoning Scores

6 fraud-relevant assessments per node

| Score                | What it Measures                                | Range                              |
|----------------------|-------------------------------------------------|------------------------------------|
| Structural Anomaly   | How unusual is the node's connectivity pattern? | 0 (typical) to 1 (extreme)         |
| Relation Consistency | How uniform is behavior across relations?       | 0 (inconsistent) to 1 (uniform)    |
| Neighborhood Risk    | How suspicious are the neighbors?               | 0 (clean) to 1 (mostly fraud)      |
| Feature Anomaly      | How statistically unusual are node features?    | 0 (normal) to 1 (extreme)          |
| Coordination Signal  | Likelihood of coordinated behavior?             | 0 (independent) to 1 (coordinated) |
| Isolation Score      | Structural isolation vs embeddedness?           | 0 (embedded) to 1 (peripheral)     |

These scores are computed ONCE during preprocessing (not during training).  
Cost: ~\$3 for Amazon, ~\$10 for Yelp. Stored as a tensor and loaded like any feature file.  
Template heuristic mode available for zero-cost approximation.

# Experimental Setup

Two datasets with different characteristics

| Property           | Amazon                  | Yelp                             |
|--------------------|-------------------------|----------------------------------|
| Nodes              | 11,944                  | 45,954                           |
| Features per node  | 25 (raw, interpretable) | 32 (pre-normalized, anonymized)  |
| Fraud rate         | 6.87%                   | 14.44%                           |
| Relations          | UPU, USU, UVU           | RUR, RTR, RSR                    |
| Train / Val / Test | 25% / 15% / 60%         | 25% / 15% / 60%                  |
| Graph density      | Moderate                | Dense (avg 149 neighbors in RSR) |

Amazon: Easier (strong feature predictors, sparser graph)

Yelp: Harder (weak anonymized features, dense noisy graph)

Testing on both validates that CAPN generalizes across different data characteristics.



# Ablation Study Design

8 systematic studies isolating each component

| Study                  | What It Tests                   | Variants                              |
|------------------------|---------------------------------|---------------------------------------|
| A1: CAPN vs CARE-GNN   | Full system comparison          | 2 (baseline vs CAPN)                  |
| A2: Label Predictor    | MLP vs linear classifier        | 3 (linear, MLP alone, MLP+policy)     |
| A3: Reward Shaping     | Shaped vs binary reward         | 4 (binary, full, dist-only, acc-only) |
| A4: LLM Priors         | Policy initialization           | 2 (no priors, LLM priors)             |
| A5: Lambda Sensitivity | Policy loss weight              | 4 (0.01, 0.05, 0.1, 0.5)              |
| A6: LLM State v1       | Sentence-transformer embeddings | 2 (base, +LLM state)                  |
| A8: LLM State v2       | Structural + reasoning scores   | 4 (base, struct, reason, both)        |

Each study changes ONE variable while keeping everything else constant.  
All studies run on both Amazon and Yelp datasets.

# Main Result: CAPN vs CARE-GNN

A1 - The headline comparison

| Metric    | CARE-GNN | CAPN             |
|-----------|----------|------------------|
| GNN AUC   | 0.9398   | 0.9445 (+0.47pp) |
| GNN AP    | 0.8535   | 0.8568 (+0.33pp) |
| GNN F1    | 0.9011   | 0.8994 (-0.17pp) |
| Label AUC | 0.8786   | 0.9306 (+5.20pp) |

Amazon Dataset

| Metric    | CARE-GNN | CAPN             |
|-----------|----------|------------------|
| GNN AUC   | 0.7660   | 0.7727 (+0.67pp) |
| GNN AP    | 0.3779   | 0.3997 (+2.18pp) |
| GNN F1    | 0.6098   | 0.5858 (-2.40pp) |
| Label AUC | 0.7304   | 0.7579 (+2.75pp) |

Yelp Dataset

CAPN consistently improves ranking metrics (AUC, AP) on both datasets. Label AUC improves dramatically (+5.20pp / +2.75pp) due to MLP predictor. F1 trade-off on Yelp: CAPN optimizes for ranking quality, which can shift the classification boundary at the default threshold.

# Component Contributions

What helps and what does not (Amazon)

| Study                   | Key Finding                                   | GNN AUC Effect     |
|-------------------------|-----------------------------------------------|--------------------|
| A2: MLP Predictor       | MLP helps label AUC; policy adds GNN AUC gain | +0.47pp (combined) |
| A3: Shaped Reward       | All shaped variants beat binary reward        | +0.47pp            |
| A4: LLM Priors          | Small help on Amazon, hurts on Yelp           | +0.08pp / -0.49pp  |
| A5: Lambda              | Very stable across all values (0.04pp range)  | Insensitive        |
| A6: LLM v1 (sent-trans) | Sentence-transformer adds no value            | -0.07pp            |
| A8: LLM v2 (reasoning)  | Claude reasoning scores help consistently     | +0.06pp            |

**Key takeaway: The policy network and shaped reward are the core contributors.**  
**LLM reasoning scores add a small but consistent boost.**  
**Sentence-transformer approach is fundamentally flawed for numerical graph data.**

# LLM Enrichment Results (A8)

Structural features vs reasoning scores

| Variant                    | GNN AUC          | GNN AP           | GNN F1           |
|----------------------------|------------------|------------------|------------------|
| CAPN baseline              | 0.9445           | 0.8568           | 0.8994           |
| + Structural (26 features) | 0.9436 (-0.09pp) | 0.8546 (-0.22pp) | 0.8671 (-3.23pp) |
| + Reasoning (6 scores)     | 0.9450 (+0.06pp) | 0.8595 (+0.27pp) | 0.9000 (+0.06pp) |
| + Both combined            | 0.9431 (-0.13pp) | 0.8548 (-0.20pp) | 0.8744 (-2.50pp) |

Why reasoning works but structural features do not:

- Structural features (26 dims) projected through MLP add noise that disrupts GNN training
- Reasoning scores (6 dims) are compact and semantically meaningful
- Each reasoning score directly captures a fraud-relevant concept
- The policy can learn clear mappings: "high neighborhood risk -> tighter threshold"

Note: These are template heuristic scores. Claude API scores (in progress) should be better.

# Cross-Dataset Analysis

Amazon vs Yelp: what generalizes?

| Finding                 | Amazon       | Yelp         |
|-------------------------|--------------|--------------|
| CAPN vs CARE-GNN (AUC)  | +0.47pp      | +0.67pp      |
| CAPN vs CARE-GNN (F1)   | -0.17pp      | -2.40pp      |
| LLM priors              | +0.08pp      | -0.49pp      |
| LLM v1 (sentence-trans) | -0.07pp      | -0.40pp      |
| Lambda sensitivity      | 0.04pp range | 0.04pp range |
| Baseline AUC            | 0.94 (easy)  | 0.77 (hard)  |

What generalizes:

- CAPN improves AUC on both datasets
- Lambda insensitivity holds on both (good for practitioners)
- Sentence-transformer fails on both

What differs:

- F1 trade-off much worse on Yelp (harder dataset)
- LLM priors help on Amazon (interpretable features) but hurt on Yelp (anonymized)

# Contributions

Summary of our work

1

Policy Network: Per-node adaptive thresholds via REINFORCE, replacing heuristic Bernoulli bandit

2

Shaped Reward: Three-component reward (distance + accuracy + regularization) for richer RL signal

3

MLP Label Predictor: +5.20pp Label AUC improvement over linear baseline

4

LLM Reasoning Integration: Novel Claude API risk scores for policy state enrichment

5

Comprehensive Ablation: 8 studies x 2 datasets providing evidence for each design choice

6

Multi-Dataset Validation: Amazon (raw features) and Yelp (anonymized) with different characteristics

# Limitations and Future Work

## Limitations

- F1 trade-off on Yelp (-2.40pp) due to ranking-optimized thresholds
- Structural features (26d) add noise rather than helping
- LLM priors hurt on anonymized datasets
- CAPN is ~7x slower than CARE-GNN (REINFORCE overhead)

## Future Work

- F1-aware reward component to address classification trade-off
- Actual Claude API reasoning scores (currently generating)
- Yelp-specific hyperparameter tuning for higher baseline
- Feature selection for structural enrichment (fewer, better features)

# Thank You

Questions?

Fady Hassanein